

October 19

Problem 1: Product of Array Except Self

Problem Statement:

Given an integer array `nums`, return an array `answer` such that `answer[i]` is equal to the product of all the elements of `nums` except `nums[i]`.

Link to problem:

<https://leetcode.com/problems/product-of-array-except-self/description/>

Example 1:

- Input: `nums = [1, 2, 3, 4]`
- Output: `[24, 12, 8, 6]`

Example 2:

- Input: `nums = [-1, 1, 0, -3, 3]`
 - Output: `[0, 0, 9, 0, 0]`
-

Solution:

```
class Solution {
    public int[] productExceptSelf(int[] nums) {
        int n = nums.length;
        int[] output = new int[n];
        int left = 1, right = 1;

        for (int i = 0; i < n; i++) {
            output[i] = left; // left product
            left *= nums[i]; // update left product
        }

        for (int i = n - 1; i >= 0; i--) {
            output[i] *= right; // multiply with right product
            right *= nums[i]; // update right product
        }

        return output;
    }
}
```

Explanation:

Two-Pass Approach: The first loop calculates the product of elements to the left of each index and stores it in the output array. The second loop calculates the product of elements to the right of each index and multiplies it with the existing value in the output array..

Time Complexity: $O(n)$

Space Complexity: $O(1)$ [excluding output array]

Problem 2: Find the Index of the First Occurrence in a String

Problem Statement:

You want to maximize your profit by choosing a single day to buy one stock and choosing a different day in the future to sell that stock. However, you can buy and sell the stock multiple times (i.e., you can make as many transactions as you like).

Link to problem:

<https://leetcode.com/problems/best-time-to-buy-and-sell-stock-ii/description/>

Example 1:

- Input: `prices = [7, 1, 5, 3, 6, 4]`
- Output: 7

Example 2:

- Input: `prices = [1, 2, 3, 4, 5]`
 - Output: 4
-

Solution:

```
class Solution {
    public int maxProfit(int[] prices) {
        int maxProfit = 0;

        for (int i = 1; i < prices.length; i++) {
            if (prices[i] > prices[i - 1]) {
                maxProfit += prices[i] - prices[i - 1];
            }
        }

        return maxProfit;
    }
}
```

```
}  
}
```

Explanation:

- The method calculates the maximum profit that can be achieved from multiple stock transactions given an array of daily prices.
- **Logic:** It iterates through the prices, adding to the total profit whenever the price on the current day is greater than the price on the previous day, representing a profitable buy-sell transaction. A variable `maxProfit` is initialized to zero to accumulate the total profit.

Example :

Let's consider the following example to illustrate how this works:

- **Input:** `prices = [7, 1, 5, 3, 6, 4]`

Step-by-Step Analysis:

- **Day 0:** Price = 7
- **Day 1:** Price = 1 (No profit, as $1 < 7$)
- **Day 2:** Price = 5 (Profit = $5 - 1 = 4$) $\rightarrow$ `maxProfit = 4`
- **Day 3:** Price = 3 (No profit, as $3 < 5$)
- **Day 4:** Price = 6 (Profit = $6 - 3 = 3$) $\rightarrow$ `maxProfit = 4 + 3 = 7`
- **Day 5:** Price = 4 (No profit, as $4 < 6$)
- **Final maxProfit:** 7

Time Complexity:

- The time complexity of the method is $O(n)$, where n is the number of days (length of the prices array). This is because we are iterating through the prices array only once.

Space Complexity:

- The space complexity is $O(1)$, as we are using a fixed amount of additional space (only a few integer variables) regardless of the input size.
-
-